

拟状结构查询

题目描述

通常使用多行的节点、**父节点**，表示一棵树，比如

西安 陕西
陕西 中国
江西 中国
中国 亚洲
泰国 亚洲

输入一个节点之后，请打印出来树中他的所有下层节点

输入描述

第一行输入行数，下面是多行数据，每行以空格区分节点和父节

接着是查询节点

输出描述

出查询节点的所有下层节点。以**字典序排序**

备注

树中的节点是唯一的，不会出现两个节点，是同一个名字

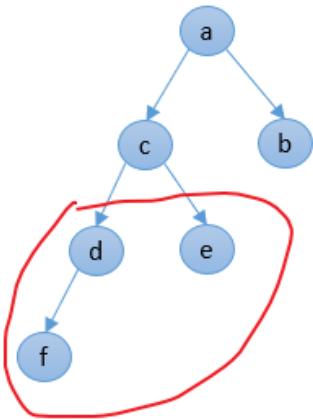
例

输入	5 b a c a
----	-----------------

	d c e c f d c
输出	d e f
说明	无

题目解析

例图 示如下



需要打印c下面的所有子节点，即d,e,f节点，按照字典序输出后是d,e,f。

本题其实就是让我们遍历树结构，可以使用深搜或者广搜。下面代码使用广搜。

关于本题树结构，可以使用Map结构实现，即key为父节点，value为子节点集合。

算法源码

```
1 | const rl = require("readline").createInterface({ input: process.stdin }); 2 | var iter = rl[Symbol.asyncIterator]();
3 | const readline = async () => (await iter.next()).value;
4 |
5 | void (async function () {
6 |     const n = parseInt(await readline());
7 |
8 |     // 构建树
9 |     const tree = {};
10 |    for (let i = 0; i < n; i++) {
11 |        const [ch, fa] = (await readline()).split(" ");
12 |
13 |        if (tree[fa] == undefined) {
14 |            tree[fa] = new Set();
15 |        }
16 |
17 |        tree[fa].add(ch);
18 |    }
19 |
20 |    // 目标节点
21 |    const target = await readline();
22 |
23 |    // 如果目标节点没有子节点，则输出空
24 |    if (!tree[target]) {
25 |        return console.log("");
26 |    }
27 |
28 |    // 记录目标节点下的所有子节点
29 |    const ans = [];
30 |
31 |    // bfs队列
32 |    const queue = [...tree[target]];
33 |
34 |    // bfs广搜
35 |    while (queue.length > 0) {
36 |        const node = queue.shift();
37 |        ans.push(node);
38 |
39 |        if (tree[node]) {
40 |            queue.push(...tree[node]);

```

```
11     }42 |     }
13
14     // 目标节点的所有子节点以字典序排序
15     ans.sort().forEach((v) => console.log(v));
16 }());
```

va算法源码

```
1  import java.util.*;
2
3  public class Main {
4      public static void main(String[] args) {
5          Scanner sc = new Scanner(System.in);
6
7          int n = sc.nextInt();
8
9          // 构建树
10         HashMap<String, HashSet<String>> tree = new HashMap<>();
11         for (int i = 0; i < n; i++) {
12             String ch = sc.next();
13             String fa = sc.next();
14
15             tree.putIfAbsent(fa, new HashSet<>());
16             tree.get(fa).add(ch);
17         }
18
19         // 目标节点
20         String target = sc.next();
21
22         System.out.println(getResult(tree, target));
23     }
24
25     public static String getResult(HashMap<String, HashSet<String>> tree, String target) {
26         // 如果目标节点没有子节点，则输出空
27         if (!tree.containsKey(target)) {
28             return "";
29         }
```

```

30 31 | // 记录目标节点下的所有子节点
32 | ArrayList<String> ans = new ArrayList<>();
33 |
34 | // bfs队列
35 | LinkedList<String> queue = new LinkedList<>(tree.get(target));
36 |
37 | // bfs广搜
38 | while (!queue.isEmpty()) {
39 |     String node = queue.removeFirst();
40 |     ans.add(node);
41 |
42 |     if (tree.containsKey(node)) queue.addAll(tree.get(node));
43 | }
44 |
45 | StringJoiner sj = new StringJoiner("\n");
46 | // 目标节点的所有子节点以字典序排序
47 | ans.stream().sorted().forEach(sj::add);
48 | return sj.toString();
49 | }
50 | }

```

python算法源码

```

1  # 输入获取
2  n = int(input())
3
4  # 构建树
5  tree = {}
6  for _ in range(n):
7      ch, fa = input().split()
8      tree.setdefault(fa, set())
9      tree[fa].add(ch)
10
11 # 目标节点
12 target = input()
13
14

```

```

15 # 算法入口16 | def getResult():
17     # 如果目标节点没有子节点，则输出空
18     if tree.get(target) is None:
19         return ""
20
21     # 记录目标节点下的所有子节点
22     ans = []
23
24     # bfs队列
25     queue = list(tree[target])
26
27     # bfs广搜
28     while len(queue) > 0:
29         node = queue.pop(0)
30         ans.append(node)
31
32         if tree.get(node) is not None:
33             queue.extend(list(tree[node]))
34
35     # 目标节点的所有子节点以字典序排序
36     return "\n".join(sorted(ans))
37
38
39 # 输出打印
40 print(getResult())

```

算法源码

```

1  #include <stdio.h>
2  #include <string.h>
3  #include <stdlib.h>
4
5  /** 链表实现，后续用于模拟队列 **/
6  typedef struct ListNode {
7      int ele;
8      struct ListNode *next;
9  } ListNode;

```

```
10 | 11 | typedef struct LinkedList {
12 |     int size;
13 |     ListNode *head;
14 |     ListNode *tail;
15 | } LinkedList;
16 |
17 | LinkedList *new_LinkedList() {
18 |     LinkedList *link = (LinkedList *) malloc(sizeof(LinkedList));
19 |     link->size = 0;
20 |     link->head = NULL;
21 |     link->tail = NULL;
22 |     return link;
23 | }
24 |
25 | void addLast_LinkedList(LinkedList *link, int ele) {
26 |     ListNode *node = (ListNode *) malloc(sizeof(ListNode));
27 |     node->ele = ele;
28 |     node->next = NULL;
29 |
30 |     if (link->size == 0) {
31 |         link->head = node;
32 |         link->tail = node;
33 |     } else {
34 |         link->tail->next = node;
35 |         link->tail = node;
36 |     }
37 |
38 |     link->size++;
39 | }
40 |
41 | int removeFirst_LinkedList(LinkedList *link) {
42 |     if (link->size == 0) exit(-1);
43 |
44 |     ListNode *removed = link->head;
45 |
46 |     if (link->size == 1) {
47 |         link->head = NULL;
48 |         link->tail = NULL;
49 |     } else {
```

```

50     link->head = link->head->next; 51     }
52
53     link->size--;
54
55     int res = removed->ele;
56
57     free(removed);
58
59     return res;
60 }
61
62 // 总节点数量
63 #define MAX_SIZE 1000
64 // 每个节点名字长度
65 #define MAX_LEN 10
66
67 // nodes[i] 记录节点名字，建立索引 i 和节点名字 nodes[i] 的映射
68 char nodes[MAX_SIZE][MAX_LEN] = {'\0'};
69 int nodes_size = 0;
70
71 // 获取名字为node的节点在nodes数组中的索引
72 int getIndex(char *node) {
73     int i = 0;
74     for (; i < nodes_size; i++) {
75         if (strcmp(nodes[i], node) == 0) break;
76     }
77
78     return i;
79 }
80
81 // 记录节点的父子关系， tree[i] 表示父节点为 i 的直接子节点集合， size = tree[i][0] 用于记录父节点 i 下面有几个直接子节点， tree[i][1] ~ tree[i][size] 对于具体子节点编号
82 int tree[MAX_SIZE][MAX_SIZE] = {0};
83
84 // 按照字典序排序节点
85 int cmp(const void *a, const void *b) {
86     int A = *(int *) a;
87     int B = *(int *) b;
88
89     return strcmp(nodes[A], nodes[B]);

```



```
90 } 91 |
92 int main() {
93     int n;
94     scanf("%d", &n);
95
96     for (int i = 0; i < n; i++) {
97         char ch[MAX_LEN], fa[MAX_LEN];
98         scanf("%s %s", ch, fa);
99
100         int j = getIndex(ch);
101         if (j == nodes_size) {
102             // 如果nodes没有收录ch节点, 则收录
103             strcpy(nodes[nodes_size++], ch);
104         }
105
106         int k = getIndex(fa);
107         if (k == nodes_size) {
108             // 如果nodes没有收录fa节点, 则收录
109             strcpy(nodes[nodes_size++], fa);
110         }
111
112         // 父节点fa的编号为k, 子节点ch的编号为j, tree[k][0] 表示节点k下有几个直接子节点
113         tree[k][++tree[k][0]] = j;
114     }
115
116     // 目标节点
117     char target[MAX_LEN];
118     scanf("%s", target);
119
120     // 如果前面没有出现过target节点 (即nodes未收录) 或者 target节点下没有子节点 (target[x][0]为0)
121     int x = getIndex(target);
122     if (x == nodes_size || tree[x][0] == 0) {
123         puts("");
124         return 0;
125     }
126
127     // 记录目标节点下面所有子节点
128     int ans[MAX_SIZE];
129     int ans_size = 0;
```

```

L30 | 131 | // bfs队列
L32 |     LinkedList *queue = new_LinkedList();
L33 |     for (int j = 1; j <= tree[x][0]; j++) {
L34 |         addLast_LinkedList(queue, tree[x][j]);
L35 |     }
L36 |
L37 | // bfs广搜
L38 | while (queue->size > 0) {
L39 |     int fa = removeFirst_LinkedList(queue);
L40 |
L41 |     ans[ans_size++] = fa;
L42 |
L43 |     for (int j = 1; j <= tree[fa][0]; j++) {
L44 |         int ch = tree[fa][j];
L45 |         addLast_LinkedList(queue, ch);
L46 |     }
L47 | }
L48 |
L49 | // 按字典序排序
L50 | qsort(ans, ans_size, sizeof(int), cmp);
L51 |
L52 | for (int i = 0; i < ans_size; i++) {
L53 |     printf("%s\n", nodes[ans[i]]);
L54 | }
L55 |
L56 | return 0;
L57 | }

```

C++算法源码

```

1 | #include <bits/stdc++.h>
2 |
3 | using namespace std;
4 |
5 | int main() {
6 |     int n;
7 |     cin >> n;

```

```

8 | 9 | // 构建树
L0 | map<string, set<string>> tree;
L1 | for (int i = 0; i < n; i++) {
L2 |     string ch, fa;
L3 |     cin >> ch >> fa;
L4 |
L5 |     tree[fa].insert(ch);
L6 | }
L7 |
L8 | // 目标节点
L9 | string target;
L0 | cin >> target;
L1 |
L2 | // 如果目标节点没有子节点，则输出空
L3 | if (tree.find(target) == tree.end()) {
L4 |     cout << "" << endl;
L5 |     return 0;
L6 | }
L7 |
L8 | // 记录目标节点下的所有子节点
L9 | vector<string> ans;
L0 |
L1 | // bfs队列
L2 | deque<string> queue;
L3 | for (const auto &node: tree[target]) {
L4 |     queue.push_back(node);
L5 | }
L6 |
L7 | // bfs广搜
L8 | while (!queue.empty()) {
L9 |     string fa = queue.front();
L0 |     queue.pop_front();
L1 |
L2 |     ans.emplace_back(fa);
L3 |
L4 |     if (tree.find(fa) != tree.end()) {
L5 |         for (const auto &ch: tree[fa]) {
L6 |             queue.push_back(ch);
L7 |         }

```

```
18         }49     }
19
20
21     // 目标节点的所有子节点以字典序排序
22     sort(ans.begin(), ans.end());
23     for (const auto &node: ans) {
24         cout << node << endl;
25     }
26
27     return 0;
28 }
```